AMD DEVELOPER HACKATHON: ACT III | TEAM: TEAM-9992 | 19 AUTOMATED TESTS

IncidentLens AMD

Evidence-Bound LLM Gateway Telemetry Triage & Root Cause Diagnostics Powered by AMD ROCm

The Core Problem

LLM gateways can suffer premature SSE termination, retry cascades, and unsafe cache writes. A model asked to triage those traces can also produce plausible explanations that are not tied to any supplied event.

The Evidence-Bound Fix

IncidentLens algorithmically parses gateway traces into an indexed evidence matrix (`[EVT-001]` , etc.), submits facts to AMD ROCm TokenFactory, and enforces an automated fail-closed citation gate on model responses.

AMD ROCm Acceleration

Native integration with AMD ROCm TokenFactory (team-9992) running `DeepSeek-V4-Flash` and `Qwen3.8-27B` on AMD GPU accelerators with real-time inference latency and token tracking.

Built by **Siddharth Srivastava** (solo builder, GitHub: `sidsri14/incidentlens-amd`)

PRODUCTION VULNERABILITIES | AI GATEWAY FAILURE MODES

Production AI Outages & The Hallucination Trap

Why trace-grounded checks matter before an AI-generated incident report is shown

⚠️ 1. Premature SSE Disconnection ("Ghost Spend")

The fixture records a client abort before the completion marker. IncidentLens flags the incomplete stream so an operator can investigate cancellation and billing behavior without inventing downstream facts.

Abort at 490 ms

MEASURED IN THE INCLUDED TRACE FIXTURE

2. Runaway Retry Cascades & 429 Storms

Clients retry failed requests immediately without randomized exponential backoff. Gateway queues saturate, compounding upstream provider rate-limits into catastrophic service paralysis.

3 HTTP 429s in 225 ms

MEASURED IN THE INCLUDED RETRY FIXTURE

3. LLM Hallucinated Incident Triage

Standard LLMs invent fictitious root causes (e.g., claiming DNS or database failure when traces show client timeouts) and recommend destructive shell commands detached from actual telemetry.

Fail-Closed Gate Required

REJECT UNCITED OR UNKNOWN REFERENCES

SYSTEM ARCHITECTURE CITATION-BOUNDED PIPELINE

The 3-Pillar Evidence-Bound Architecture

Deterministic trace parsing, stable evidence indexing, and fail-closed citation checks

1 Deterministic Telemetry Parser

`src/trace_parser.mjs` parses raw SSE stream transitions, HTTP response codes, and socket lifecycles algorithmically:

- Flags premature disconnections before `[DONE]` sentinels.
- Detects retry storms with delta < 200ms without jitter.
- Identifies attempted cache writes on truncated streams.
- Calculates TTFT & latency SLA outlier thresholds.

2 AMD ROCm Evidence Matrix

`src/evidence_matrix.mjs` converts parsed events into a stable, inspectable evidence table:

- Every trace event receives a canonical ID: `[EVT-001]`, `[EVT-002]`.
- Timestamps, durations, token counts, and error codes are bound.
- Only validated evidence bundles are dispatched to the AMD ROCm inference engine.

3 Fail-Closed Citation Gate

`src/citation_validator.mjs` inspects every sentence in the model's incident report before surfacing to SREs:

- Every root-cause claim must cite a valid `[EVT-xxx]` ID.
- Any uncited factual assertion is flagged and rejected.
- Any hallucinated event ID not in the evidence matrix causes immediate fail-closed denial.

HARDWARE ACCELERATION AMD TOKENFACTORY

AMD ROCm Infrastructure Integration

Live inference on AMD GPU accelerators via official AMD Developer Program team-9992

AMD TokenFactory Client (`src/amd_rocm_client.mjs`)

Direct connectivity to AMD's high-performance ROCm inference endpoint:

- **Endpoint:** `https://developer.amd.com.cn/radeon/api/v1/chat/completions`
- **Team Account:** `team-9992` (authorized by AMD AI Developer Program)
- **Runtime:** Event-issued TokenFactory endpoint; provider metadata is recorded from the API response.
- **Models:** `DeepSeek-V4-Flash` (fast triage) & `Qwen3.8-27B` (deep audit)
- **Strict Evidence Prompting:** Enforces event ID citation requirements directly in system instructions.

Verified Live Benchmark

19,614 ms
END-TO-END AMD ROCM INFERENCE LATENCY

809 Tokens
EVIDENCE SYNTHESIS & TRIAGE PLAN GENERATED

5 Event Citations
FIVE KNOWN EVENT REFERENCES ACCEPTED BY THE GATE

USER EXPERIENCE OPERATOR COCKPIT

Interactive Operator HUD & Failure Simulator

Real-time trace waterfall, anomaly visualization, and automated citation badges

🖥️ Operator Cockpit Features

- **Preset Scenario Simulators:** One-click generation of Ghost Spend, Retry Storms, Cache Inconsistencies, or Clean Baselines.
- **Telemetry Waterfall Table:** Inspects status codes, socket duration, TTFT, token counts, and anomaly badges.
- **Evidence Matrix Viewer:** Real-time display of indexed events with deterministic event IDs.
- **Citation Verification Badge:** Visual confirmation that claim units cite known IDs, or that the response was rejected.

🛡️ Security & Enterprise Hardening

- **Explicit Deployment Modes:** Local server binds to `127.0.0.1`; the public demo runs a separate no-key serverless evaluation path.
- **Content Security Policy:** Strict CSP headers, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`.
- **Bounded Input:** JSON-only requests are capped at 64 KiB and parsed into a constrained event schema.
- **Zero-Key Fallback:** Includes deterministic offline fixtures for instant evaluation without external credentials.

ENGINEERING RIGOR PUBLIC OSS PROTOTYPE

Verification, CI/CD & Open Source Delivery

Reproducible tests, multi-version CI, and a public MIT repository

✅ Automated Test Suite

19/19 passing tests using the native Node.js test runner (`node --test`):

- `test/trace_parser.test.mjs` : Ghost spend, retry storms, cache hazards, latency outliers.
- `test/citation_validator.test.mjs` : Known-ID acceptance plus unknown and uncited claim rejection.
- `test/server.test.mjs` : Scenario dispatch and local HTTP analysis lifecycle.
- `test/vercel_handler.test.mjs` : No-key serverless mode and truthful mode labelling.

🚀 GitHub Actions CI Matrix

GitHub Actions Run #38590710694:

- **Linux (Ubuntu-latest):** Node 20 (Pass), Node 22 (Pass), Node 24 (Pass).
- **Windows (Windows-latest):** Node 20 (Pass), Node 22 (Pass), Node 24 (Pass).
- **Security Audit:** `npm audit` reports 0 vulnerabilities.
- **Public MIT Repo:** github.com/sidsri14/incidentlens-amd

Run locally: `npm test && npm start`